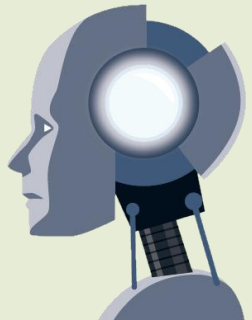
# 03

# 컴퓨터 구조

단원 목차

3.1 폰 노이만의 저장 프로그램
3.2 다양한 기억장치
3.3 중앙처리장치 CPU
3.4 프로그램 실행 과정
3.5 마이크로프로세서

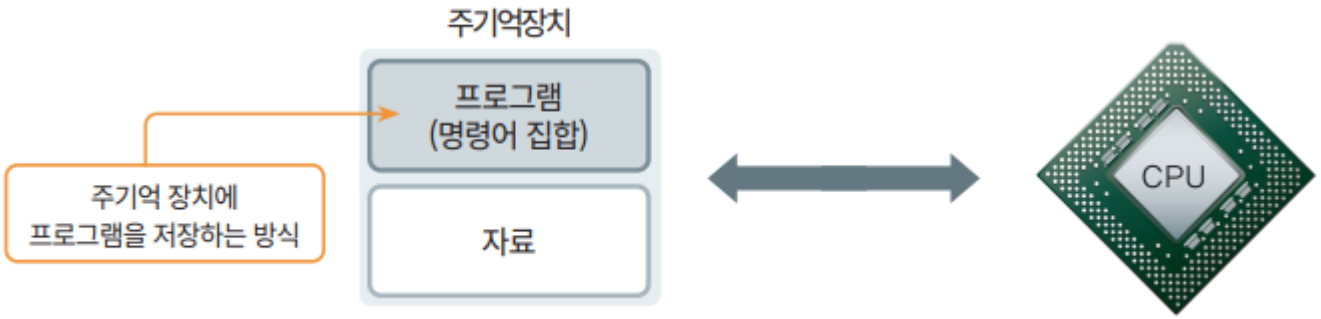


# 프로그램 내장 방식

메모리에 자료 뿐만 아니라 프로그램도 저장하는 프로그램 내장 방식

- **저장 프로그램(Stored Program) 방식**
  - 폰 노이만이 고안
    - **메모리에 자료와 프로그램이 함께 저장**
- **중앙처리장치(CPU)**
  - 메모리에서 필요한 자료를 이용
  - 저장된 명령어를 순차적(Sequential)으로 실행

에니악 컴퓨터와 같은 이전 방식은 작업을 할 때마다 전기회로를 바꾸어야 했지만 프로그램 내장방식은 주기억장치에 프로그램만 바꾸어 저장하면 원하는 프로그램을 수행할 수 있다.



# 명령어 형식

메모리에 저장되며, 프로그램을 구성하는 명령어의 형식과 종류

- **명령어(instruction) 구성**
  - 연산 부분(operation part)과 피연산 부분(operand part)
    - **연산 부분은 명령어가 수행해야 할 기능을 의미하는 코드**
    - **피연산 부분은 연산에 참여하는 자료를 의미하는 코드**
- **명령어가 16비트로 구성**
  - 4비트는 연산 종류(opcode)
  - 12비트는 피연산자의 메모리 주소(address)
  - 피연산자 수는 없거나 2개 또는 3개
    - **메모리 주소 또는 레지스터**

그림 4.2 ▸ 16비트의 명령어 형식 예

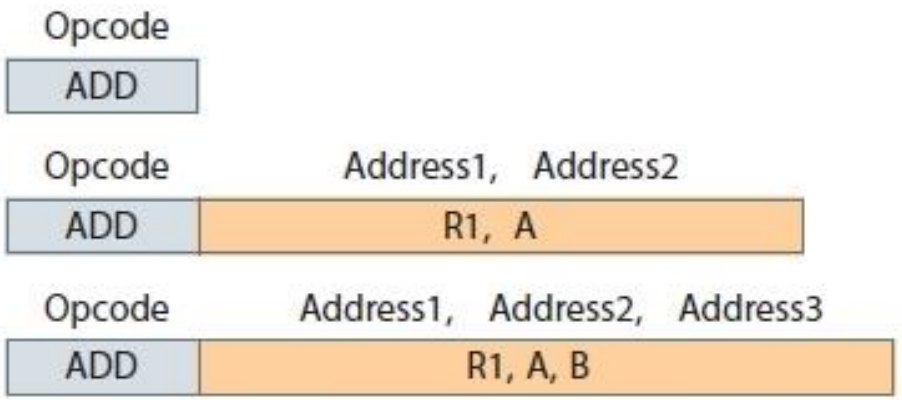



그림 4.3 ▸ 피연산자의 수에 따른 다양한 명령어 형식

# 명령어 종류

명령어의 연산자는 인간에게 친숙하게 표기하기 위하여 ADD(add), HLT(halt) 등의 단어를 이용

- **연산자**
  - ADD(add)
  - LDA(load address)
  - STA(store address)
  - HLT(halt) 등의 기호 단어를 이용
- **피연산자**
  - A, B, C 등으로 기술

표 4.1 명령어 종류와 구문 형식, 기능

| 명령어 | 구문 형식 | 기능 |
|---|---|---|
| ADD | ADD A | 피연산자의 자료 A와 레지스터의 자료를 더하는 명령어 |
| LDA | LDA B | 피연산자의 자료 B(주소 B의 자료)를 레지스터에 가져오는 명령어 |
| STA | STA C | 레지스터의 내용을 피연산자 C(주소 C)에 저장하는 명령어 |
| HLT | HLT | 컴퓨터를 종료하는 명령어 |

# 명령어 예

**두 수를 더해서 메모리에 저장**하고 프로그램을 종료하는 Assembly 언어

- **LDA 100 : 메모리 100번지에서 값을 누산기로**
- **ADD 101 : 메모리 101번지의 값을 누산기에 더함**
- **STA 102 : 결과를 메모리 102번지에 저장**
- **HLT : 프로그램 종료**

# 명령어 예

100 + 200

메모리 주소 10에는 100이 저장되어 있음
메모리 주소 11에는 200이 저장되어 있음
결과를 저장할 곳은 메모리 주소 12

LDA 10 : 메모리 10번지의 값(100)을 누산기로 로드
ADD 11 : 메모리 11번지의 값(200)을 누산기에 더함 (결과: 300)
STA 12 : 결과 300을 메모리 12번지에 저장
HLT : 프로그램 종료

# 저급 언어

사람에게 친숙하지 않아 이해하기 어려우나 컴퓨터에게 작업을 지시할 수 있는 언어

- **기계어**
  - 컴퓨터를 작동시키기 위해 0과 1로 나타낸 컴퓨터 고유 명령 형식 언어
- **어셈블리어**
  - 컴퓨터 명령어인 기계어를 사람이 일상 생활에서 사용하는 자연 언어와 유사하게 만든 언어
  - 명령어는 연산자와 피연산자를 몇 개의 문자 조합으로 기호화

그림 4.4 ▸ 기계어의 구성

그림 4.5 ▸ 명령에 따른 어셈블리어와 기계어

# 주기억장치: 주소

중앙처리장치가 해야 할 작업 내용인 프로그램 명령어와 프로그램에서 이용할 자료를 저장

- **주기억장치의 구조**
  - 주소
    - **메모리의 저장소는 주소(address)를 이용하여 각각 바이트 단위로 고유하게 식별**
    - **컴퓨터가 한 번에 작업할 수 있는 데이터의 단위를 워드**
      - **워드는 32비트 또는 64비트**

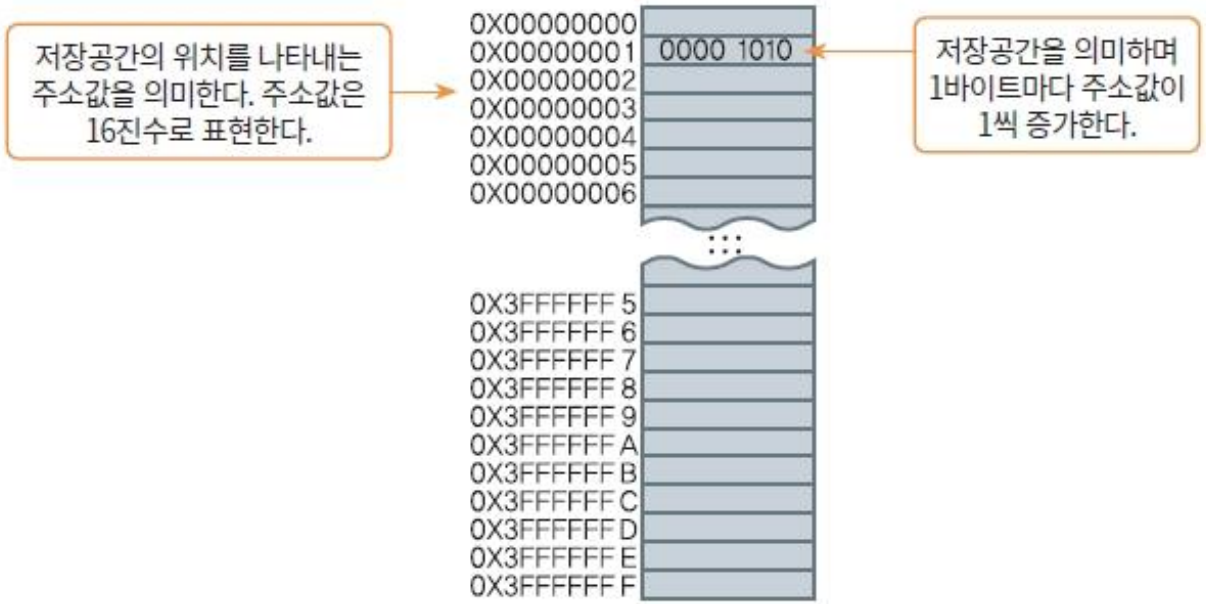



그림 4.6 ▸ 메모리와 주소

# 버스

CPU와 메모리 또는 입출력 장치 간의 정보 전달을 위한 통로(버스)

- **주기억장치의 구조**
  - 주소
  - 버스
    - **관련 자료 전달 경로**
      - **주소버스, 자료버스, 제어버스**

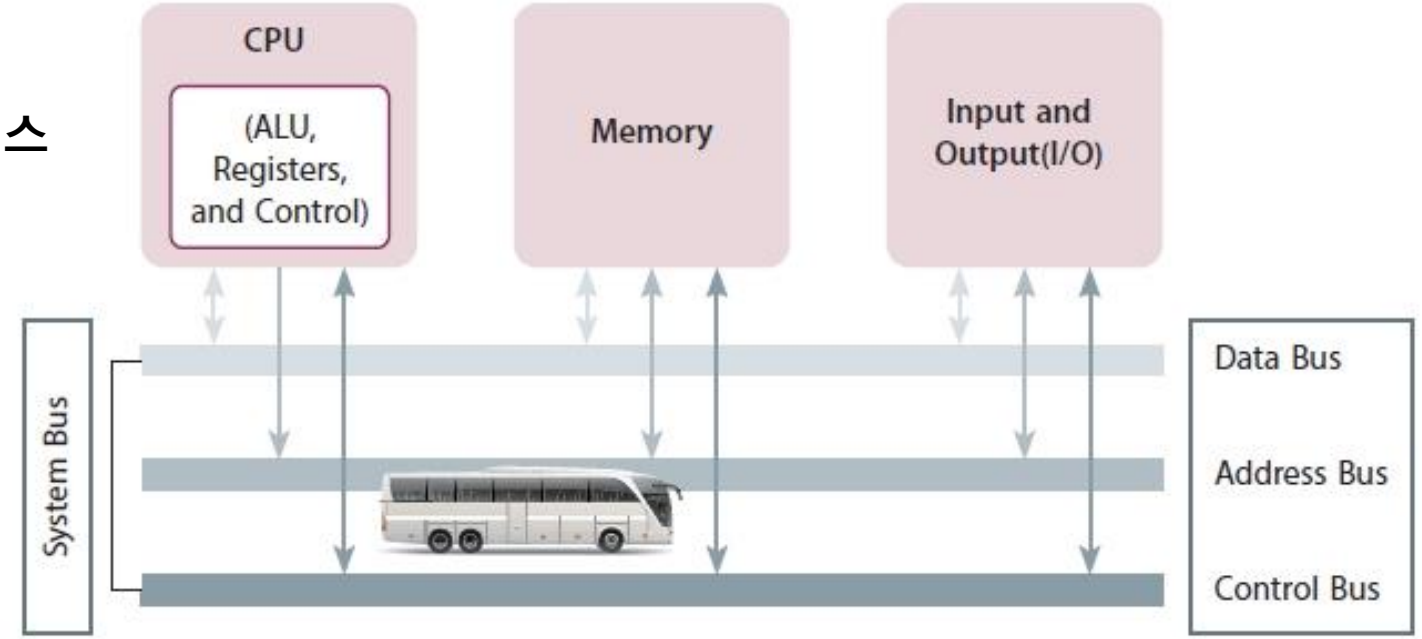



그림 4.7 ▸ 메모리와 중앙처리장치, 입출력장치를 연결하는 시스템 버스

# 버스

CPU와 메모리 또는 입출력 장치 간의 정보 전달을 위한 통로(버스)

- **주소버스 (Address Bus)**
  - 역할: CPU가 **어디(메모리 주소)**에 접근할지를 지정함
  - 단방향: CPU ➡ 메모리/장치

- **자료버스 (Data Bus)**
  - 역할: 실제 데이터(값)를 전달함
  - 양방향: CPU ↔ 메모리/장치

- **제어버스 (Control Bus)**
  - 역할: 동작 제어 신호를 주고받음 (읽기/쓰기/인터럽트 등)
  - 양방향

| 버스 종류 | 비유 |
| --- | --- |
| 주소버스 | 택배 주소 (어디로 보낼지) |
| 자료버스 | 택배 물건 (실제 데이터) |
| 제어버스 | 택배 지시서 (보내기/받기, 긴급 등) |

# 주기억장치 RAM

유일한 숫자인 주소를 사용하여 메모리의 특정 위치의 내용을 바로 참조

- **Random Access Memory 임의 접근 메모리**
  - 소멸성(volatile) 기억장치
  - 쓰기와 읽기의 두 회로가 있어서 정보의 쓰기와 읽기가 가능
- **DRAM과 SRAM , VRAM(GPU에 사용)**
  - DRAM은 전원이 연결된 상태에서 일정한 주기마다 전기적으로 재충전 필요
    - **주기억장치로 주로 사용(SDRAM)**
      - **SDRAM(Synchronous DRAM)은 시스템 클럭(clock)과 동기를 맞춘 DRAM**
  - SRAM은 전원만 연결되어 있으면 정보가 지워지지 않는 기억장치
    - **캐쉬 메모리(cache memory)에 주로 사용**

그림 4.8 ▸ RAM

# RAM 작동 흐름 예시

- **예: CPU가 메모리에서 데이터를 읽고 싶을 때**
  1. 주소버스에 원하는 메모리 주소를 보냄
  2. 제어버스에 "읽기(Read)" 신호를 보냄
  3. RAM이 그 주소의 데이터를 자료버스를 통해 CPU로 보냄
  4. CPU는 그 데이터를 사용

# 기억장치 ROM

한 번 저장된 자료는 더 이상 쓰기를 할 수 없이 읽기만 가능한 메모리

- **읽기 전용 메모리, Read Only Memory**
  - ROM, 비소멸성(nonvolatile) 메모리
    - **ROM도 메모리에 임의 접근이 가능**
- **종류**
  - Mask ROM
    - **자료를 써넣는 방식에 따라 기억된 데이터를 지우거나 변경할 수 없음**
  - PROM(Programmable ROM)
    - **임의의 프로그램을 기억**
  - EPROM(Erasable Programmable ROM)
    - **자외선 또는 X선 등을 이용하여 데이터를 지우거나 새로운 데이터를 입력**

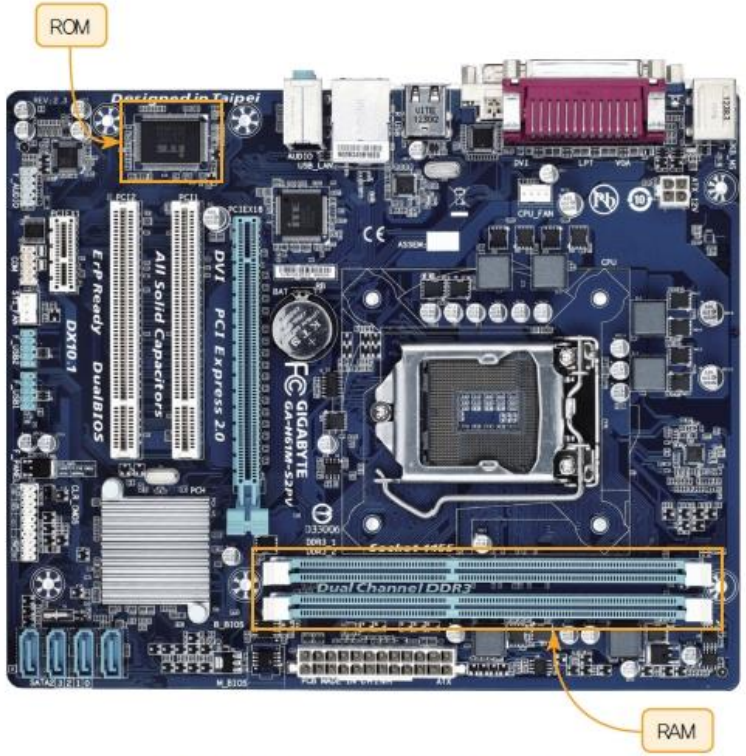



그림 4.9 ▸ 마더보드에서의 ROM과 RAM

# BIOS

바이오스(BIOS: Basic Input/Output System)는 컴퓨터를 시작한 후 운영체제가 시스템을 제어하기 이전까지 하드웨어를 점검하여 컴퓨터를 실행

- **바이오스(BIOS: Basic Input/Output System)**
  - 컴퓨터의 부팅을 위한 펌웨어(firmware)
  - 컴퓨터의 전원이 연결되면 바이오스는 먼저 POST 실행
- **POST(Power-On Self Test)**
  - 일련의 시동자체시험 과정
  - 주기억장치와 보조기억장치, 컴퓨터 키보드, 그래픽 카드 등, 주변장치 연결 확인
  - 관련 하드웨어의 이상 여부를 확인하는 초기화(initialize) 작업 수행
  - 보조기억장치의 연결에 문제가 있거나 보조기억장치 내의 운영체제 파일에 문제가 있다면
    - **바이오스는 부팅 불가 메시지를 출력**
  - POST 과정에서 아무 문제가 없다면
    - **운영체제를 컴퓨터의 메모리인 RAM에 로드(load)**

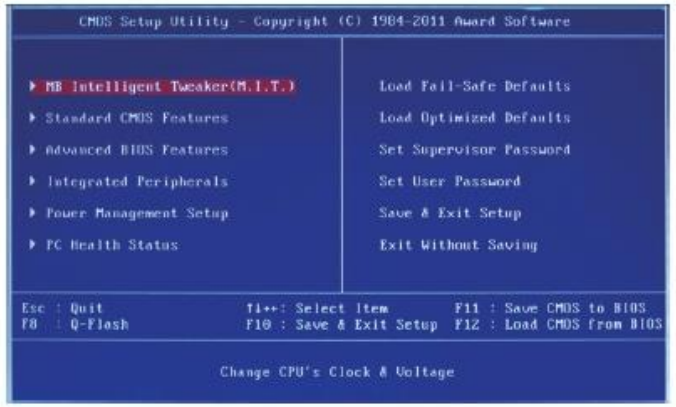



그림 4.10 ▸ 바이오스 화면과 바이오스가 저장된 ROM

# 캐쉬 메모리

주변기기의 속도를 빠르게 하는 대표적인 방법이 캐시(cache) 메모리의 사용

- **캐쉬의 사용 이유**
  - CPU에 비해 상대적으로 주변기기의 속도가 매우 느림
  - **주기억장치와 CPU의 속도의 차이를** 해결
  - 캐시 메모리는 메인 메모리보다 대개 약 10배쯤 더 빠름
    - **저장 속도가 빠르고 고가인 SRAM을 이용**

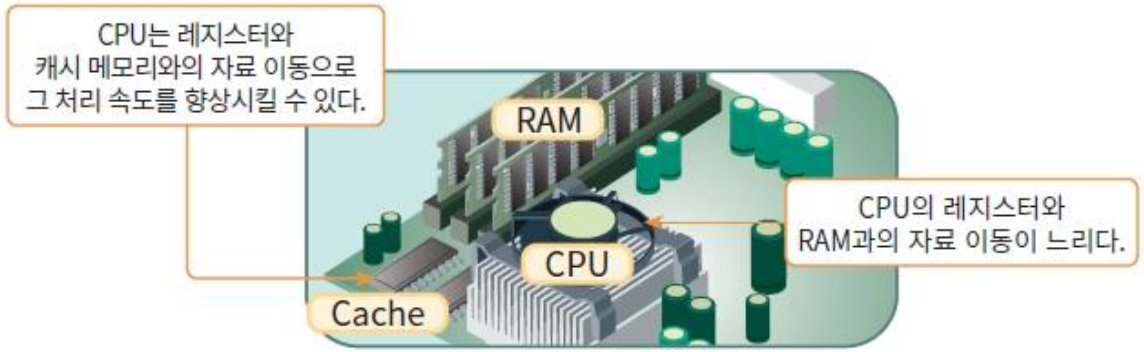



그림 4.12 ▸ CPU와 RAM 사이의 캐시 메모리

- **캐쉬의 종류**
  - 수준1 캐쉬, 수준2 캐쉬
  - 디스크 캐쉬
    - **RAM과 디스크 사이에 일정량의 임시메모리**

그림 4.13 ▸ 캐시의 종류

# 버퍼

버퍼링(buffering)은 버퍼를 활용하는 방식 또는 버퍼를 채우는 동작

- **버퍼**
  - 속도의 차이가 나는 두 장치 간에 효율적인 작업 처리를 위한 임시 저장 공간
- **버퍼의 필요성**
  - 프로세서의 속도는 입출력 주변장치의 속도보다 훨씬 더 빠름
    - **프로세서는 주변장치의 응답을 기다리다 시간을 허비할 수 있음**
  - 이를 방지하기 위해
    - **RAM의 일부를 임시저장 공간인 버퍼(buffer) 메모리로 사용**
      - **입출력장치로 전달될 데이터를 일시적으로 저장**
      - **그동안 프로세서는 다른 요청이나 작업을 계속 수행 가능**

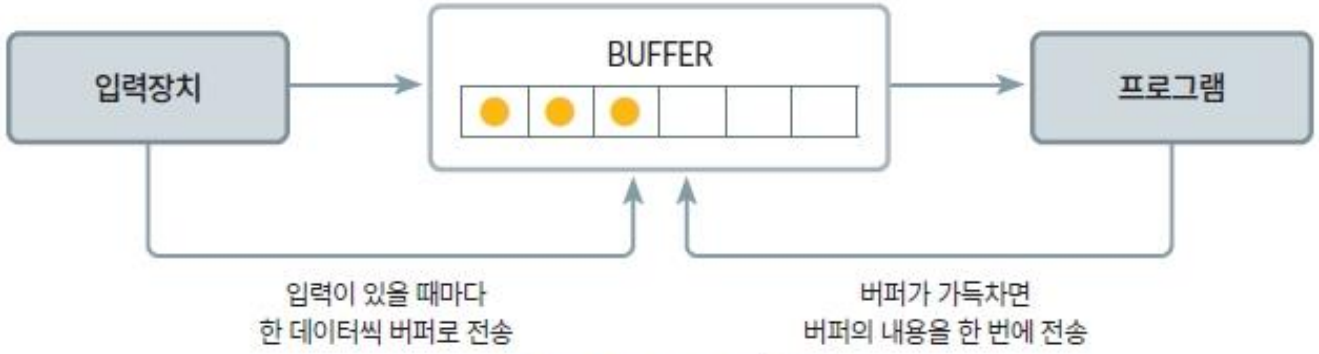



그림 4.14 ▸ ROM의 종류

# 보조기억장치

주기억장치보다 가격이 싸며, 전원 없이도 대용량의 자료를 영구적으로 저장할 수 있는 기억장치

- **보조기억장치(secondary memory unit)**
  - CPU 외부에 위치
    - **주기억장치의 제한된 기억용량을 보조**
  - 순차접근(sequential access) 방식
    - **순차적으로 접근이 가능**
      - **자기 테이프**
  - 직접접근(direct access) 방식
    - **원하는 위치에 바로 쓰고 읽을 수 있는 직접 접근이 가능**
      - **자기 디스크와 자기 드럼**
- **HDD: Hard Disk Drive**
  - 헤드를 이용하여 여러 개의 원형 알루미늄 기판인 디스크에 자료를 저장하는 방식

하드디스크 하드디스크의 상세이미지

그림 4.15 ▸ 하드디스크 드라이브

# HDD를 보완하는 SSD

빠르게 확산되고 있는 차세대 대용량 저장장치이며, 가격이 계속 하락하면서 많은 컴퓨터에 내장

- **SSD**
  - 플래시메모리와 이를 제어하는 컨트롤러로 구성된 대용량 저장장치
  - SSD의 장점과 활용
    - **HDD와 비교해 읽고 쓰는 속도가 빠르며 전력 사용량이 적고 충격에 강하며 발열과 소음도 적음**
      - **SSD에 운영체제와 자주 사용하는 프로그램을 설치**
      - **HDD에는 영화나 음악 같은 대용량의 자료를 저장하여 함께 사용**

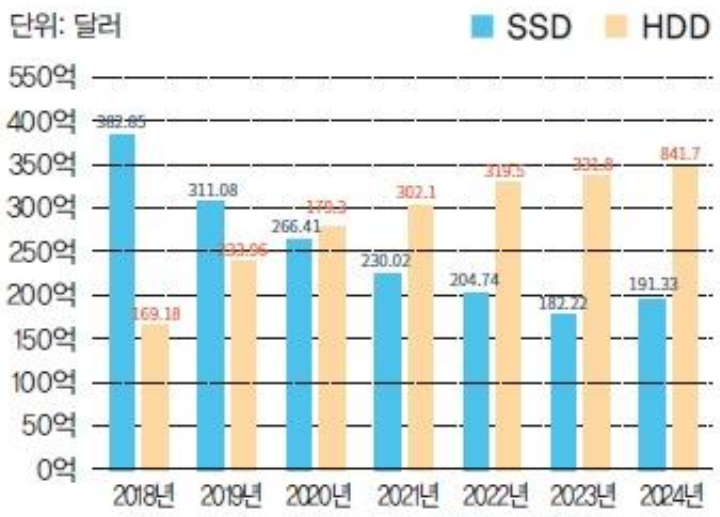


그림 4.16 ▸ SSD와 HDD의 시장 규모 예측

# 휴대가 간편한 외장하드와 USB 메모리

USB와 같은 인터페이스를 통해 직접 컴퓨터와 연결할 수 있으므로 휴대와 이동이 편리하다는 특징

- **외장하드**
  - 12TB 이하가 2.5인치 제품으로 출시
  - 대부분의 제품이 USB 인터페이스로 컴퓨터와 연결
- **플래시 메모리**
  - RAM과 ROM의 장점을 합친 메모리
  - 소비전력이 적고, 비소멸성 메모리
    - **전원이 꺼지더라도 저장된 정보가 사라지지 않음**
  - 보조 기억장치
    - **디지털 텔레비전, 디지털 캠코더**
    - **휴대전화, 디지털 카메라**
    - **개인휴대단말기(PDA), 게임기**
    - **MP3 플레이어, USB 메모리**
- **USB 메모리**
  - 휴대용으로 가장 많이 사용
  - TV, 자동차 등에도 연결이 가능

그림 4.18 ▸ 다양한 플래시 메모리

# 저장장치의 계층

레지스터, 주기억장치, 보조기억장치 등 다양한 종류의 저장장치를 이용하는 이유는?

- **기억장치 계층의 필요**
  - 저장장치의 속도와 용량, 가격과 그 쓰임새를 고려
  - 저장장치의 속도가 빠르면
    - **가격이 상승**
    - **동일한 비용으로 속도를 유지하려면**
      - **용량은 작아져야 함.**
- **다양한 저장 장치의 이용**
  - 보조기억장치에 저장
    - **앞으로 이용하려는 프로그램이나 자료**
  - 주기억장치에 저장
    - **현재 실행 중인 프로그램이나 자료**
  - 캐시 메모리에 저장
    - **현재 집중적으로 이용되는 프로그램이나 자료**
  - 레지스터에 저장하여 연산에 직접 이용
    - **연산이 필요한 프로그램이나 자료**

그림 4.20 ▸ 기억장치 계층 구조

# 중앙처리장치

메모리에 저장된 프로그램과 자료를 이용하여 실제 작업을 수행하는 회로 장치

- **CPU: Central Processing Unit**
- **주요 구성 요소**
  - 연산장치: 자료의 연산을 수행
  - 제어장치: 컴퓨터의 작동을 제어
  - 레지스터: 연산에 필요한 자료를 임시로 저장
  - 버스: 자료버스, 제어버스

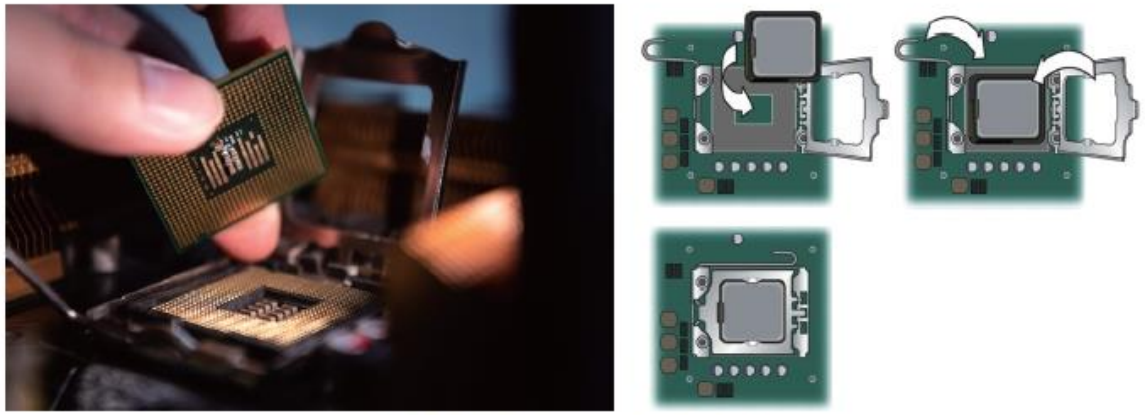


그림 4.22 ▸ 마더보드에 CPU를 장착

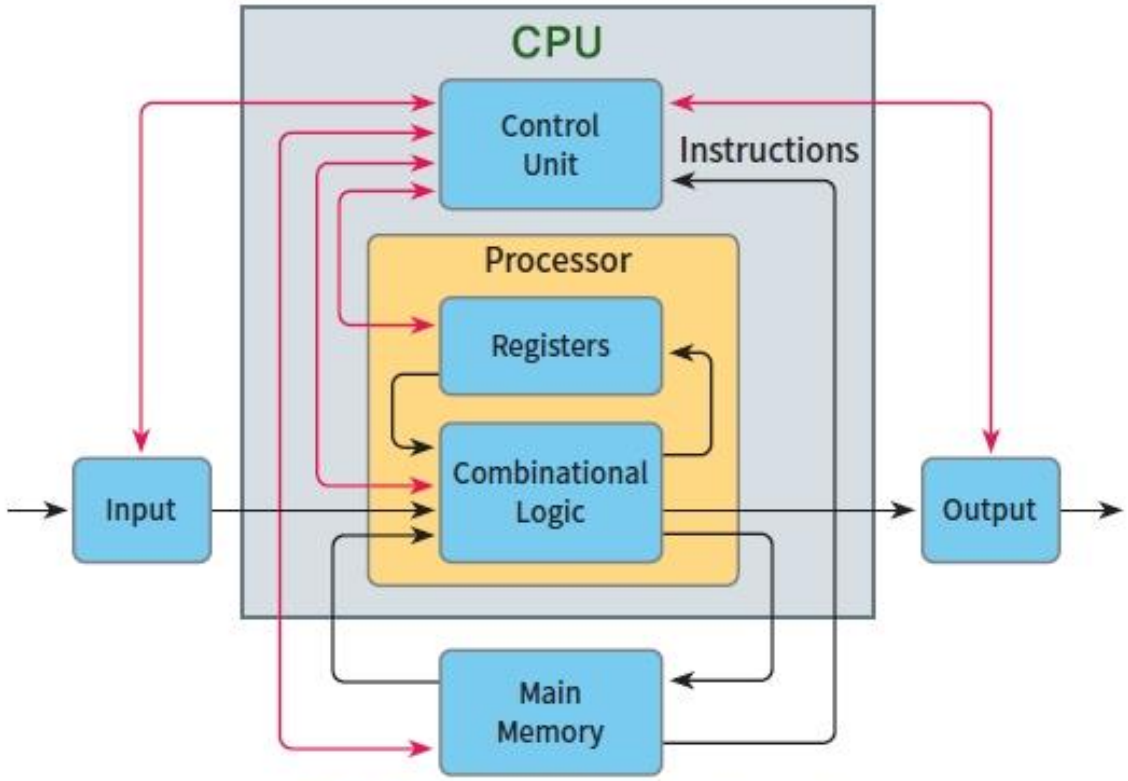



그림 4.21 ▸ 중앙처리장치(CPU) 구성

# 연산장치

연산장치(ALU: Arithmetic and Logic Unit)는 산술연산과 논리연산을 수행하는 회로

- **레지스터의 이용**
  - 다음을 연산에 참여할 피연산자로 이용
    - **중앙처리장치의 임시기억장소인 누산 레지스터(Accumulator)와 자료 레지스터(Data Register)에 저장된 자료**
  - 결과는 다시 누산 레지스터에 저장되어 필요하면 주기억장치에 저장되거나 다른 연산에 이용
    - **AC <- AC + DR**
  - 두 레지스터 피연산자의 연산을 연산장치가 제어장치의 신호를 받아 실행

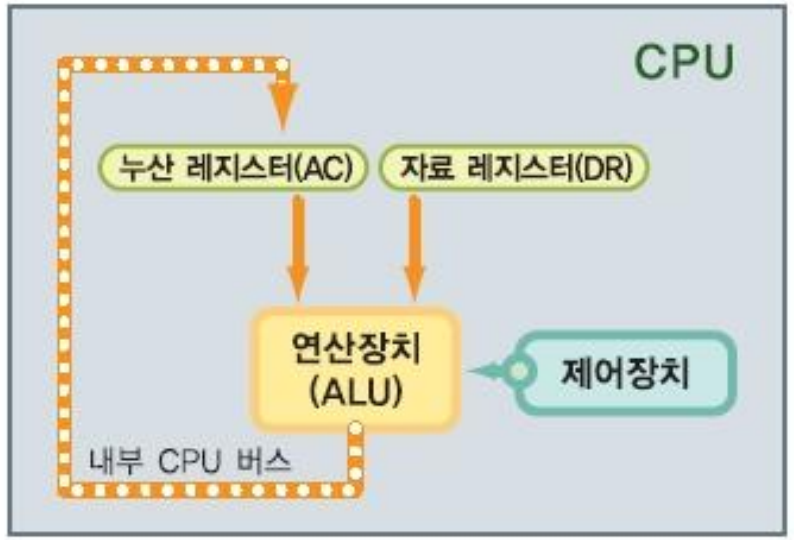



그림 4.23 ▸ CPU 내부의 연산장치

# 제어장치

신체의 여러 부분을 제어하는 인간의 뇌와 같이 제어장치는 중앙처리장치에서 연산을 제어

- **Control Unit**
  - 산술 및 논리 연산에 요구되는 작업을 연속적으로 수행하는 신호를 보냄으로써 연산장치와 레지스터가 명령을 수행하게 하는 장치
    - **인간의 뇌와 같은 요소**
- **구성**
  - 여러 개의 해독기(decoder)와 제어기로 구성

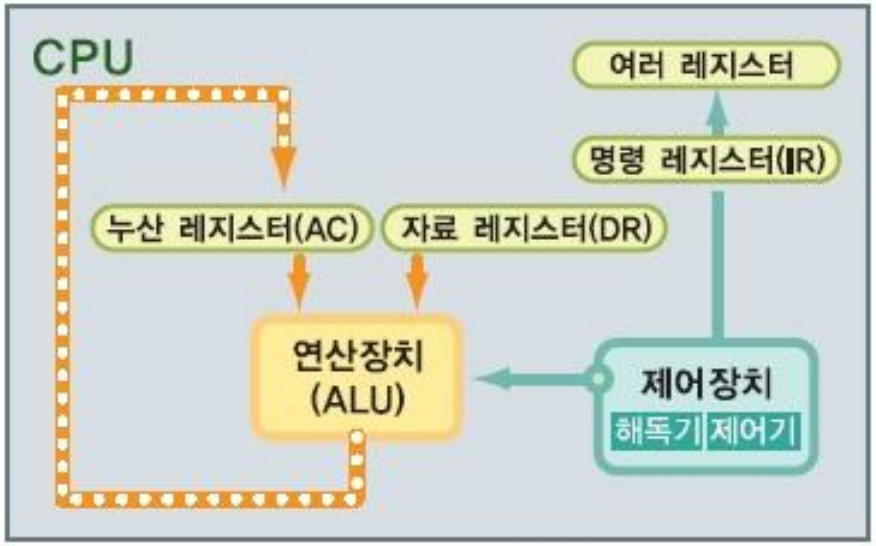



그림 4.25 ▸ 제어장치

# 레지스터

중앙처리장치 내부처리 연산에 필요한 다양한 임시 기억장소

- **여러 개의 레지스터를 가짐**
  - 중앙처리장치는 컴퓨터가 명령을 수행하는 과정을 처리하기 위해
  - 중앙처리장치 내의 레지스터 크기와 수는 중앙처리장치의 성능에 매우 중요한 요소이므로 가격과 성능을 고려하여 결정

표 4.2 주요 레지스터의 종류와 기능

| 레지스터 심볼 | 레지스터 이름 | 기능 |
|---|---|---|
| DR | 자료 레지스터 (Data Register) | 연산에 필요한 피연산자를 저장하는 레지스터 |
| AR | 주소 레지스터 (Address Register) | 현재 접근할 기억장소의 주소를 기억하는 레지스터 |
| AC | 누산 레지스터 (Accumulator Register) | 연산장치의 입출력 데이터를 임시적으로 기억하는 레지스터 |
| IR | 명령어 레지스터 (Instruction Register) | 현재 수행 중인 명령어를 저장하고 있는 레지스터 |
| PC | 프로그램 카운터 (Program Counter) | 다음에 실행할 명령어의 메모리 주소가 저장된 레지스터 |
| TR | 임시 레지스터 (Temporary Register) | 임시로 자료를 저장하는 레지스터로 범용 레지스터라고도 부름 |

# 명령어 처리 과정

CPU와 주기억장치 간의 다양한 자료 전송이 발생

- **명령어 처리과정**
  - 주기억장치의 명령어와 자료가 중앙처리장치의 여러 임시 저장장소인 레지스터로 전송
  - 명령어를 처리한 후 다시 처리 결과인 자료가 주기억장치로 전송되는 과정을 거침

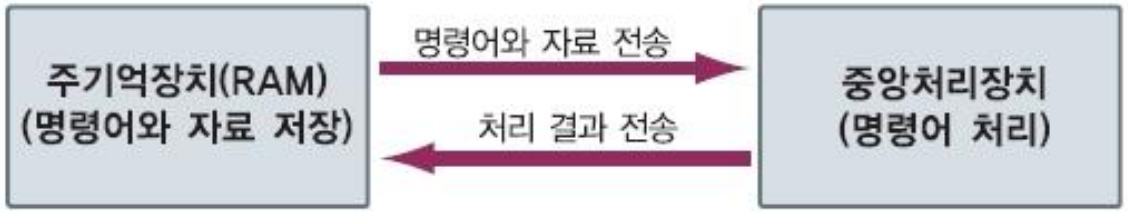



그림 4.26 ▸ CPU와 주기억장치와의 자료 교환

# 기계주기 1/2

명령어의 집합인 중앙처리장치는 하나의 명령어를 실행하기 위하여 인출(fetch), 해독(decode), 실행(execution)의 세 과정을 거침

- **하나의 명령어를 실행**
  - 중앙처리장치
    - **인출(fetch) 또는 추출**
    - **해독(decode)**
    - **실행(execution)의 세 과정을 거침**

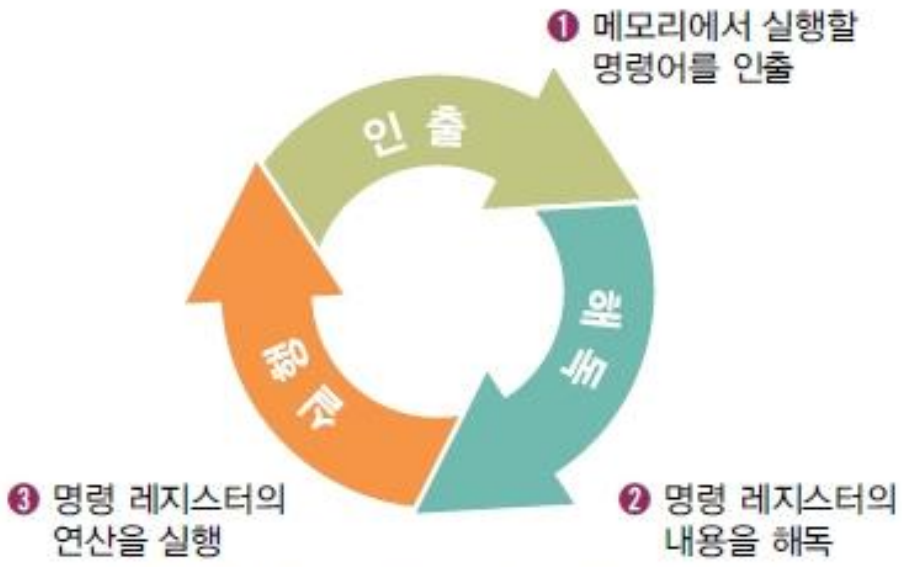



그림 4.27 ▸ 명령어 실행 과정인 기계 주기

# 기계주기 2/2

명령어의 집합인 프로그램을 실행하려면 3 과정인 기계주기를 계속적으로 반복

- **인출, 해독, 실행의 세 과정**
  - 인출 단계, fetch
    - **제어 장치가 프로그램 카운터에 있는 주소로 다음에 수행할 명령어를 명령 레지스터에 저장**
    - **이후 다음 명령어를 수행하기 위해서 프로그램 카운터를 하나 증가 시킴**
  - 해독 단계, decode
    - **제어 장치는 명령어 레지스터에 있는 명령어를 연산 부분과 피연산 부분으로 해독**
    - **명령어가 피연산부분이 있는 명령어라면**
      - **피연산 부분의 메모리 주소를 주소 레지스터에 저장**
  - 실행 단계, execution
    - **중앙처리장치는 각 구성 요소에게 작업 지시를 내림**

- **하나의 명령어 실행이 종료되면 프로그램 카운터가 가리키는 다음 명령어를 가지고 다시 기계 주기를 반복**

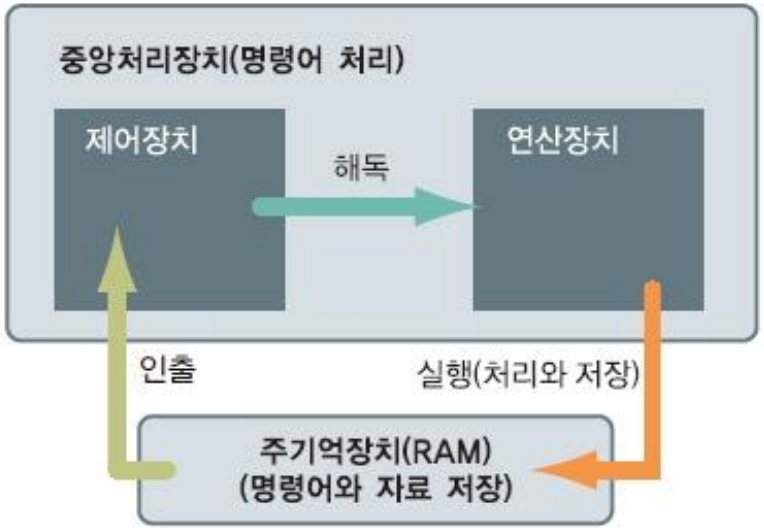



그림 4.28 ▸ 기계 주기와 자료의 이동

# 두 정수 합 구하기 1/2

고급 언어로 기술된 C <- A + B, 두 수의 합 구하기

- **만일 두 수가 각각 32와 -18이라면**
  - 기호 A는 32를 의미하며, 기호 B는 -18을 의미
  - 메모리에 더 작은 단위의 여러 명령어 집합으로 구성하여 그 명령을 실행
  - 두 정수의 합을 구하기 위해서는 다음과 같이 4개의 명령어 집합으로 가능
- **명령어 LDA의 기능**
  - 주소 레지스터(AR)의 주소 값을 갖는 메모리 자료(M[AR])를 누산 레지스터(AC)에 저장
  - 이 처리를 위하여 자료 레지스터(DR)를 다시 누산 레지스터에 저장

표 4.3 두 정수의 합을 위한 명령어 집합

| 순서 | 명령어 | 의미 |
|---|---|---|
| 명령어1 | LDA A | 메모리 A의 내용을 누산 레지스터(AC)에 저장 |
| 명령어2 | ADD B | 메모리 B의 내용과 누산 레지스터(AC)의 값을 더하여 누산 레지스터(AC)에 다시 저장 |
| 명령어3 | STA C | 누산 레지스터(AC)의 값을 메모리 C에 저장 |
| 명령어4 | HLT | 프로그램 종료 |

# 두 정수 합 구하기 2/2

고급 언어로 기술된 C <- A + B, 두 수의 합 구하기

- **명령어 LDA의 기능**
  - 주소 레지스터(AR)의 주소 값을 갖는 메모리 자료(M[AR])를 누산 레지스터(AC)에 저장
  - 이 처리를 위하여 자료 레지스터(DR)를 다시 누산 레지스터에 저장

표 4.4 명령어의 세부 수행 기능

| 명령어 | 세부 수행 기능 표시 | 세부 수행 기능 의미 |
|---|---|---|
| LDA | DR ← M[AR]<br>AC ← DR | • 주소 레지스터(AR)의 주소값의 메모리 자료(M[AR])를 자료 레지스터(DR)에 저장<br>• 자료 레지스터(DR)를 다시 누산 레지스터(AC)에 저장 |
| ADD | DR ← M[AR]<br>AC ← AC + DR | • 주소 레지스터(AR)의 주소값의 메모리 자료(M[AR])를 자료 레지스터(DR)에 저장한 후<br>• 누산 레지스터(AC)와 자료 레지스터(DR)를 더하여 그 결과를 다시 누산 레지스터(AC)에 저장 |
| STA | M[AR] ← AC | • 누산 레지스터(AC)의 값을 주소 레지스터(AR)의 주소값 위치의 메모리(M[AR])에 저장 |

# 작업 명령어와 메모리 이해

LDA A, ADD B, STA C, HLT의 명령어 수행

- **피연산자**
  - 피연산자 A: 메모리 주소 0012FF40에 저장된 32
  - 피연산자 B: 마찬가지로 명령어 ADD B에서 메모리 주소 0012FF44에 저장된 -18
  - 피연산자 C:
    - **명령어 STA C에서 피연산자 C는 메모리 주소 0012FF48을 가리키며**
    - **여기에는 32 + (-18)의 결과인 14가 저장**

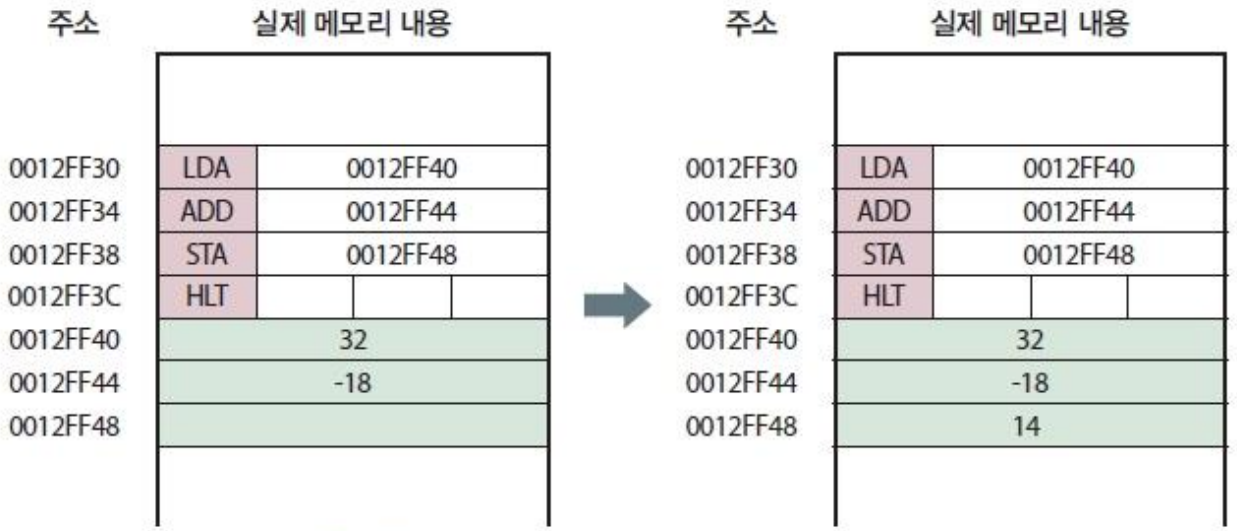



그림 4.29 ▸ 명령어 실행 전과 실행 후의 메모리 내부

# 자료 버스 폭과 병목 현상

컴퓨터가 한 번에 작업할 수 있는 데이터의 크기를 워드 크기(word size)라 함

- **자료버스 폭**
  - 연산장치와 레지스터 등과 같은 CPU의 내부 구성 요소 간에 자료를 전달하는 통로의 비트 수
  - 워드 크기(word size)
    - **레지스터의 크기**
- **'32비트 컴퓨터', '64비트 프로세서'**
  - 프로세서 내에 있는 레지스터의 크기
  - 자료 버스 폭은 마이크로프로세서의 워드 크기와 비례하며 워드 크기와 같거나 더 큼
- **병목현상**
  - 레지스터는 64비트인데 자료 버스는 32비트라면 연산은 64비트 단위로 수행
  - 한 번에 자료를 전달할 수 없으므로 병목 현상이 발생

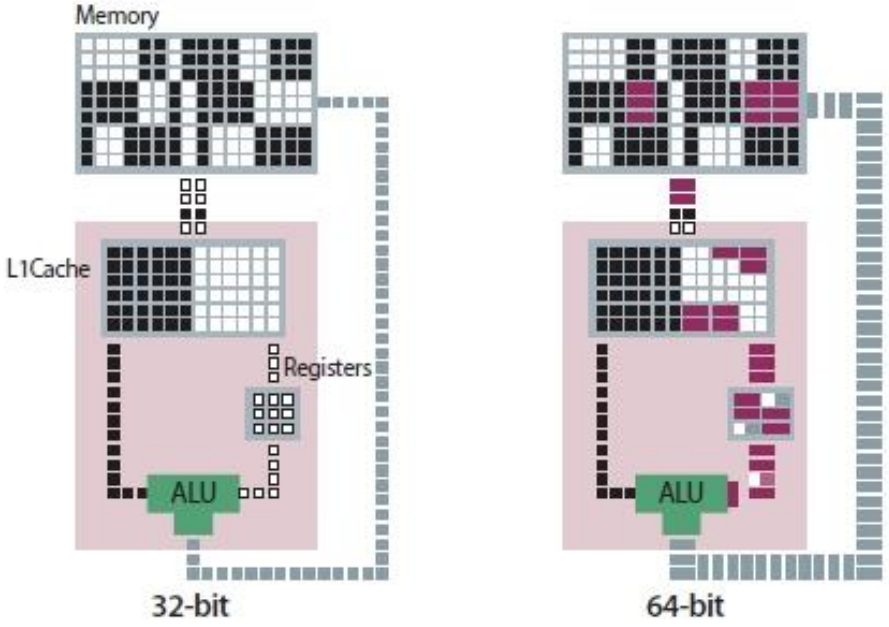



그림 4.30 ▸ 32비트와 64비트 컴퓨터 비교

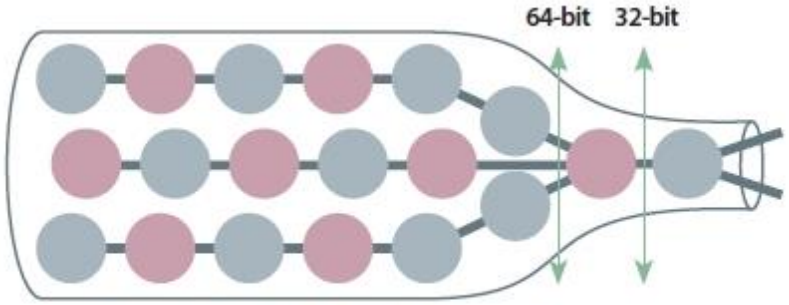



그림 4.31 ▸ 병목 현상

# 마이크로프로세서 성능

사이클당 연산 수와 자료 버스의 폭, 레지스터의 수와 크기, 그리고 캐시 메모리의 크기 등으로 결정

- **자료버스 폭**
- **클럭 속도**
  - 클럭 속도의 단위인 Hz
    - **1초당 진동의 반복 횟수를 재는 단위**
  - 클럭 속도는 연산 속도와 비례
    - **프로세서는 하나의 명령어를 특정 수의 클럭 사이클에서 실행**
- **병렬 처리**
  - 하나의 컴퓨터에서 2개 이상의 CPU를 이용
    - **한 번에 여러 개의 명령어를 동시에 실행**
- **컴퓨터의 성능**
  - CPU의 성능과 RAM의 용량
  - 외부 자료 버스의 크기

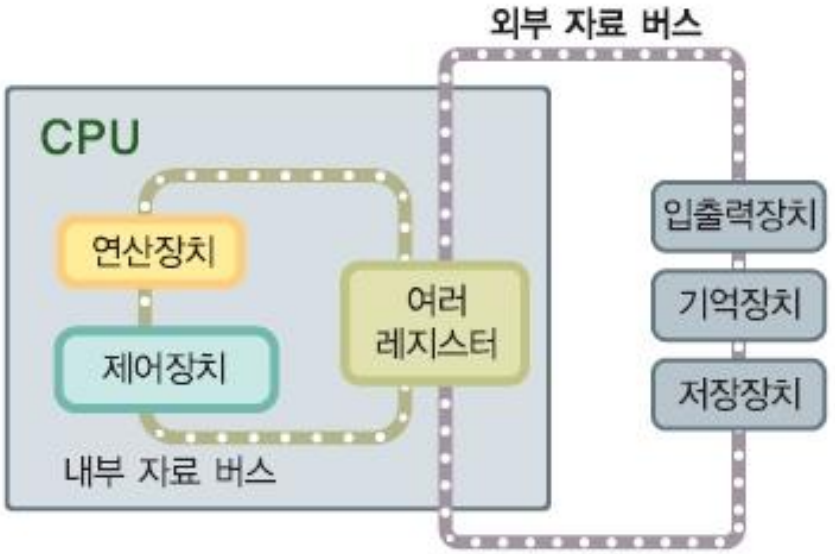



그림 4.32 ▸ 내부자료 버스와 외부자료 버스

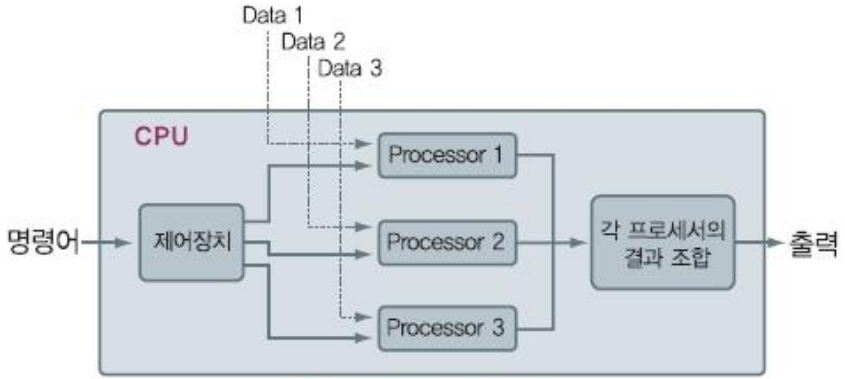



그림 4.33 ▸ 병렬처리

# 인텔의 마이크로프로세서

1971년 하나의 칩으로 된 컴퓨터(Computer On a Chip)라 부르는 마이크로프로세서 4004를 처음 출시

- **인텔**
  - 마이크로프로세서를 생산하는 대표적 기업
  - IBM이 IBM 호환 PC에 인텔의 8088 프로세서를 탑재하여 출시하면서 인텔은 눈부시게 성장
  - 펜티엄
    - **1993년에 인텔은 64비트 자료 버스를 사용한 마이크로프로세서인 펜티엄을 발표**
  - 아이테니엄
    - **2001년에는 고성능 서버용으로 진정한 의미의 64비트 시대를 연 프로세서를 출시**
  - 코어 i3, i5, i7, i9 CPU를 출시
    - **2011년 이후**

표 4.5 인텔 계열 마이크로프로세서의 발전

| 발표 연도 | 명칭 | 버스 폭 | 클럭 속도 | 특징 |
|---|---|---|---|---|
| 1971 | 4004 | 4비트 | 740 KHz | |
| 1974 | 8080 | 8비트 | 2 MHz | |
| 1979 | 8088 | 16비트 | ~8 MHz | 16비트형 XT |
| 1982 | 80286 | 16비트 | ~ 12 MHz | 16비트형 AT |
| 1985 | 80386 | 32비트 | ~ 33 MHz | 32비트형 PC |
| 1989 | 80486 | 32비트 | ~ 100 MHz | |
| 1993 | Pentium | 64비트 | ~ 200 MHz | |
| 1995 | Pentium Pro | 64비트 | 200 MHz ~ | |
| 1997 | Pentium MMX | 64비트 | 233 MHz ~ | |
| 1998 | Pentium II | 64비트 | 233 MHz ~ | |
| 1998 | Celeron | 64비트 | 400 MHz ~ | |
| 1999 | Pentium III | 64비트 | 450 MHz ~ | |
| 2000 | Pentium IV | 64비트 | 1.3 GHz ~ | |
| 2001 | Itanium | 64비트 | 800 MHz ~ | PC 계열에서 64비트 시대를 개척 |
| 2002 | Itanium 2 | 64비트 | 900 MHz ~ | |
| 2005 | Pentium D | 64비트 | 2.0 GHz ~ | |
| 2006 | Core | 64비트 | 1.8 GHz ~ | |
| 2006 | Core 2 Duo | 64비트 | 2.6 GHz ~ | |
| 2007 | Core 2 Quad | 64비트 | 2.4 GHz ~ | |
| 2008 | Core i7 | 64비트 | 2.5 GHz ~ | |
| 2009 | Core i5 | 64비트 | 2.4 GHz ~ | |
| 2010 | Core i3 | 64비트 | 2.9 GHz ~ | |
| 2011 이후 | 2, 3, 4세대 Core i3, Core i5, Core i7 | 64비트 | 2.5 GHz ~ | |
| 2013 | Core i7 Extreme Edition | 64비트 | 4.0 GHz ~ | |
| 2018 | Core i9 | 64비트 | 4.8 GHz | |

# 무어의 법칙

인텔의 공동 설립자인 고든 무어(Gorden Moore)가 1965년도에 한 연설에서 유래

- **"마이크로 칩의 처리 능력은 18개월마다 두 배로 증대된다"**
  - 실제로 마이크로프로세서의 성능은 약 18개월에서 24개월마다 두 배로 증가
  - 무어의 법칙은 앞으로 몇 년 안에 물리학의 근본 원리와 상충되는 어려움 때문에 지켜지지 않을지도 모름
    - **인텔을 비롯한 마이크로프로세서 생산 업체는 이러한 무어의 법칙을 유지하기 위해**
  - 포괄적인 무어의 법칙
    - **마이크로 칩의 처리 능력에만 국한되지 않고 정보기술 분야의 발전이 빠르게 진행되고 있다는 의미로 포괄적으로 이해**

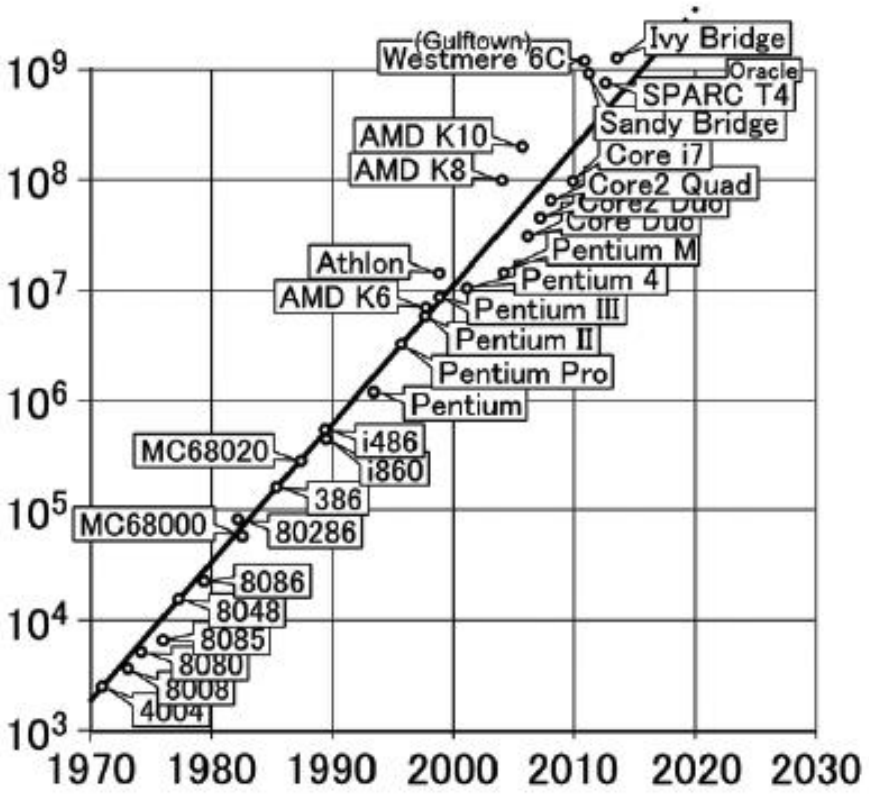



그림 4.35 ▸ 무어의 법칙에 따른 마이크로프로세서의 발전

# 무어의 법칙과 마이크로프로세서 성능의 한계

급변하는 정보기술 환경과 미세 공정 기술의 한계, 그리고 급증하는 투자 비용으로 인해 18개월마다 반도체의 용량을 2배 씩 늘리는 일은 점점 어려워지고 있는 실정

- **무어의 법칙**
  - 저장 용량이 1년 6개월마다 2배 씩 커진다는 법칙
- **무어의 법칙에 한계론**
  - 2016년 국제반도체기술로드맵(ITRS) 위원회
    - **반도체 기술의 미세화 기술이 2021년 이후에는 더 이상 지속될 수 없다는 사실을 공식적으로 선언**
    - **순수한 성능보다 그래픽 기능이나 전력 효율 향상에 중점**
- **긍정론**
  - 반도체 공정 및 디자인 부분에 혁신이 필요한 시기이며 이러한 어려움은 극복
  - 미국 국가과학재단(National Science Foundation)은 '무어의 법칙 이면의 과학 및 엔지니어링'(Science and Engineering behind Moore's Law)이라는 이름의 프로젝트
    - **제조, 나노기술, 다중 코어칩, 양자 컴퓨팅 등의 새로운 기술 연구를 재정적으로 지원**
  - 앞으로 몇 년 후에 무어의 법칙이 들어맞지 않더라도 계속된 노력으로 다른 측면의 기술은 발전

# 프로세서 분류: CISC

고유한 명령어 집합을 제공하는 마이크로프로세서는 CISC 계열과 RISC 계열로 구분

- **마이크로프로세서는 프로세서마다 고유한 명령어 집합을 제공**
  - CISC(Complex Instruction Set Computing) 계열
    - **명령어는 크게 복합 명령어 집합으로 구성된**
  - RISC(Reduced Instruction Set Computing) 계열
    - **축소 명령어 집합으로 구성**
- **CISC: Complex Instruction Set Computing**
  - 복합 명령어 집합 컴퓨팅계열
    - **명령어의 구조가 복잡하고 100-250개의 다양한 명령어를 제공**
    - **인텔의 80x86 계열과 모토롤라의 680x0 계열의 프로세서**
    - **복잡한 연산을 하나의 명령어로 처리하려는 의도에서 시작**
      - **다양한 길이를 가지며 메모리의 자료를 직접 참조하는 연산도 많이 제공**
- **장단점**
  - 복잡한 프로그램을 적은 수의 명령어로 구성할 수 있는 장점
  - 복잡한 명령어의 실행을 위한 복잡한 회로가 이용
    - **생산가가 비싸고 전력 소모가 많아 열이 많이 발생하는 단점**

# 프로세서 분류: RISC

RISC 명령어는 고정 길이이며 쉽게 해독이 되는 명령어 형식

- **RISC: Reduced Instruction Set Computing**
  - 축소 명령어 집합 컴퓨팅 계열
    - **명령어의 수가 적고 그 구조도 단순**
    - **레지스터 내부에서 모든 연산이 수행되며 메모리의 참조는 제한적**
    - **상대적으로 레지스터가 많은 특징**
  - 1988년 중반 애플의 매킨토시에 장착된 모토롤라의 PowerPC에서 처음 구현된 RISC 프로세서
    - **이후 Sun, HP, NEC의 워크스테이션 컴퓨터**
- **장점**
  - 전체적으로는 CISC 프로세서보다 수행 속도가 빠름
  - 하나의 프로그램을 수행
    - **CISC보다 많은 명령어를 실행해야 하지만**
    - **하나의 명령어가 단순하여 그 처리 속도가 매우 빠름**

# CISC vs RISC 요약

| 항목 | CISC | RISC |
|---|---|---|
| 명령어 수 | 많고 복잡한 명령어 포함 | 단순하고 적은 명령어 사용 |
| 명령어 길이 | 가변 길이 | 고정 길이 |
| 실행 속도 | 한 명령어에 많은 작업 → 느릴 수 있음 | 명령어당 처리 속도 빠름 |
| 하드웨어 복잡도 | 복잡함 | 간단함 |
| 대표 CPU | 인텔 x86, AMD 등 | ARM, MIPS, RISC-V 등 |
| 메모리 접근 | 명령어 내에서 직접 메모리 접근 가능 | 보통 명령어는 레지스터 간 연산만 수행 |
| 사용 | 일반 PC/노트북(x86/x64) | 모바일, 저전력 디바이스(ARM계열) |